

Lab4 Group 8

Liuxi Mei

Xiaochen Liu

2025-02-18

Question 1: Consider a random variable X with the following probability density function:

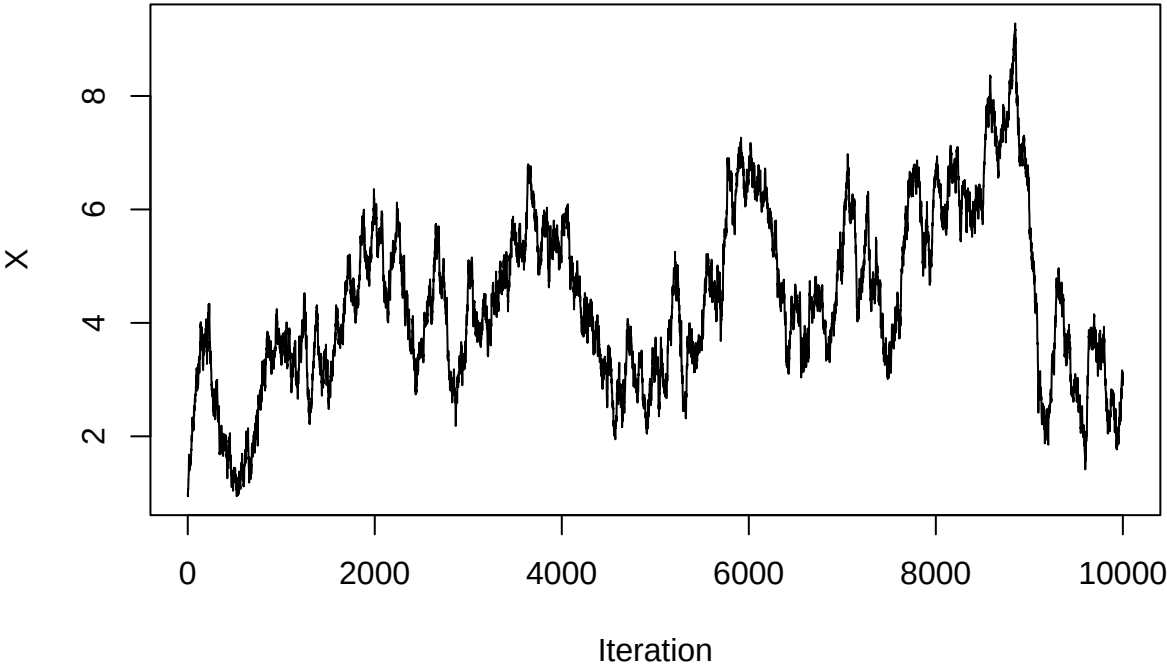
$$f(x) = 120x^5 e^{-x}, \quad x > 0$$

Part a

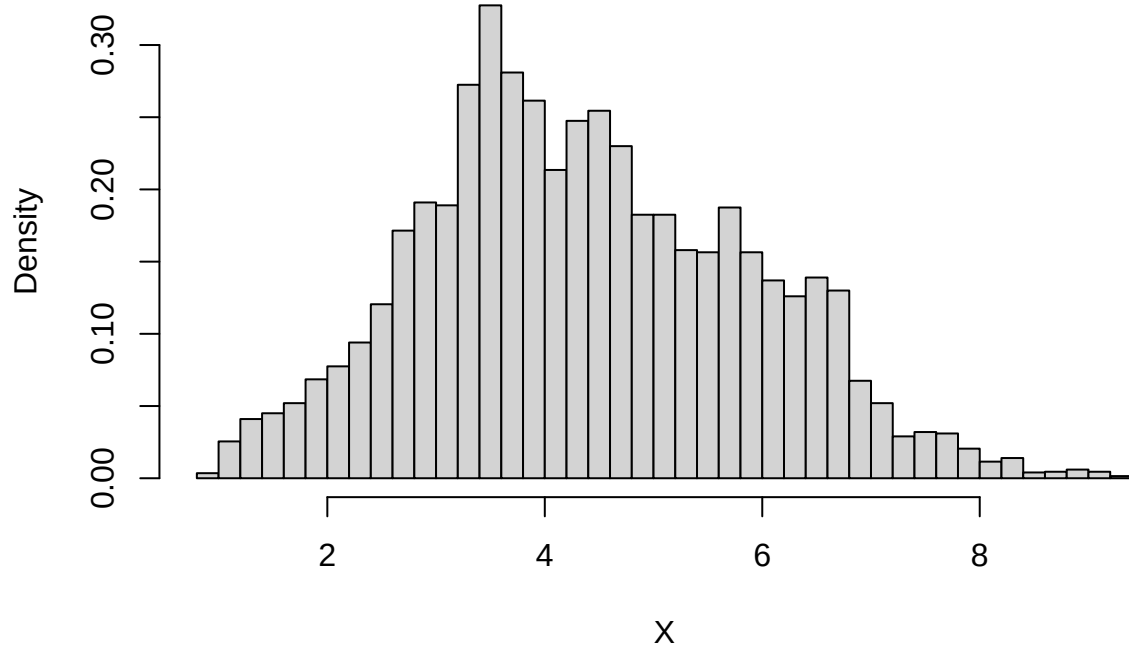
Use the Metropolis–Hastings algorithm to generate 10,000 samples X_t , $t = 1, \dots, 10,000$, from this distribution. Use a normal distribution with mean X_t and standard deviation 0.1 as the proposal distribution when you generate X_{t+1} ; take some starting point. Plot the chain you obtained with iterations on the horizontal axis. What can you guess about the convergence of the chain? If there is a burn-in period, what can be the size of this period? What is the acceptance rate? Plot a histogram of the sample.

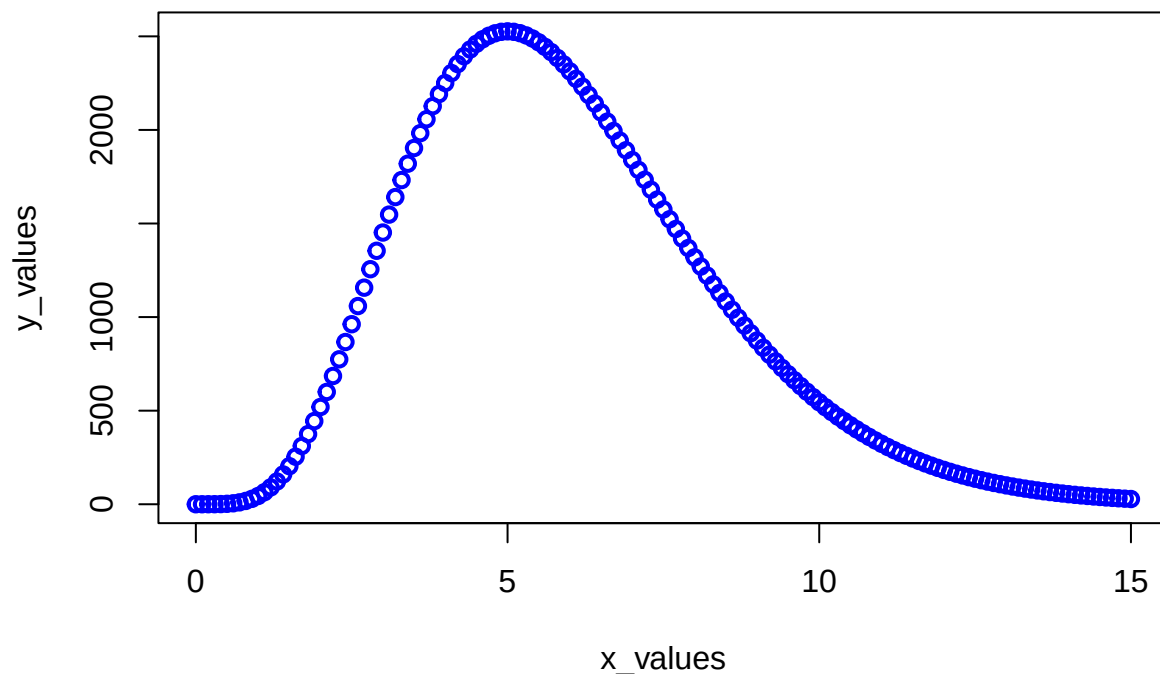
```
##          [,1]
## chain 1.0000000
## x      0.9439524
## x      1.0629731
## x      1.0700239
## x      1.0591273
## x      1.1052190
```

Metropolis-Hastings Chain



Histogram of Samples with Target Density





```
## Acceptance Rate: 0.9856
```

It seems that the convergence is not reached for this chain. The plot of X-values vs iteration number still has a great fluctuation and the plotted histogram does not have a good fitting with the true plot of target function. The X-values of histogram of sample lies in $[0,10]$ with the highest density at around 4. While the true function plot gives v-values in $[0,15]$ and it peaks around 5.

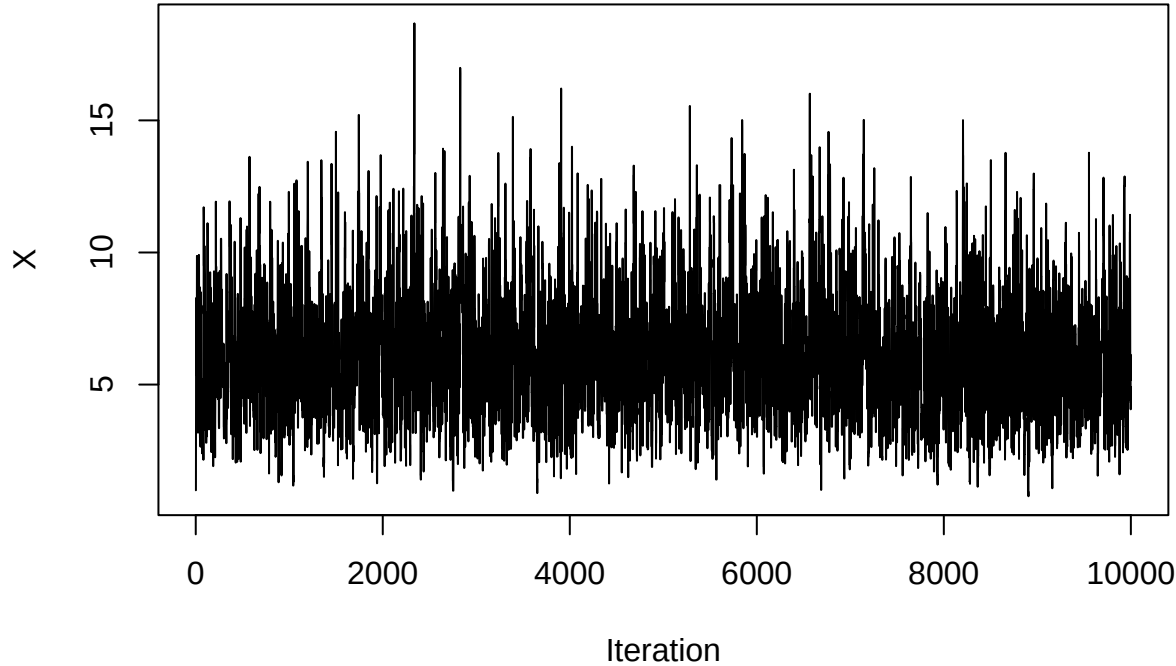
By looking into the first 100 samples, we found that X-value start to stabilize around 2 from the 40th sampling. Therefore, 40 can be the size of burn-in period if there is a burn-in period. The acceptance rate is calculated to be 0.9856.

Part b

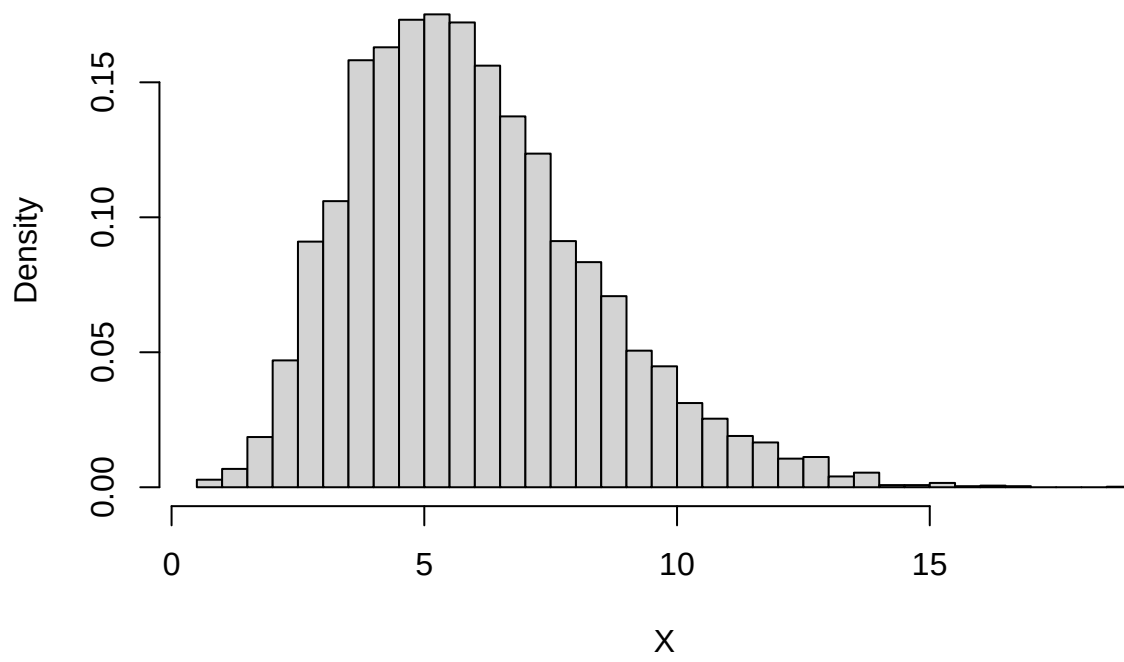
Perform Part a by using the chi-square distribution $\chi^2(\lfloor X_t + 1 \rfloor)$ as a proposal distribution, where $\lfloor x \rfloor$ is the floor function, meaning the integer part of x for positive x , i.e. $\lfloor 2.95 \rfloor = 2$.

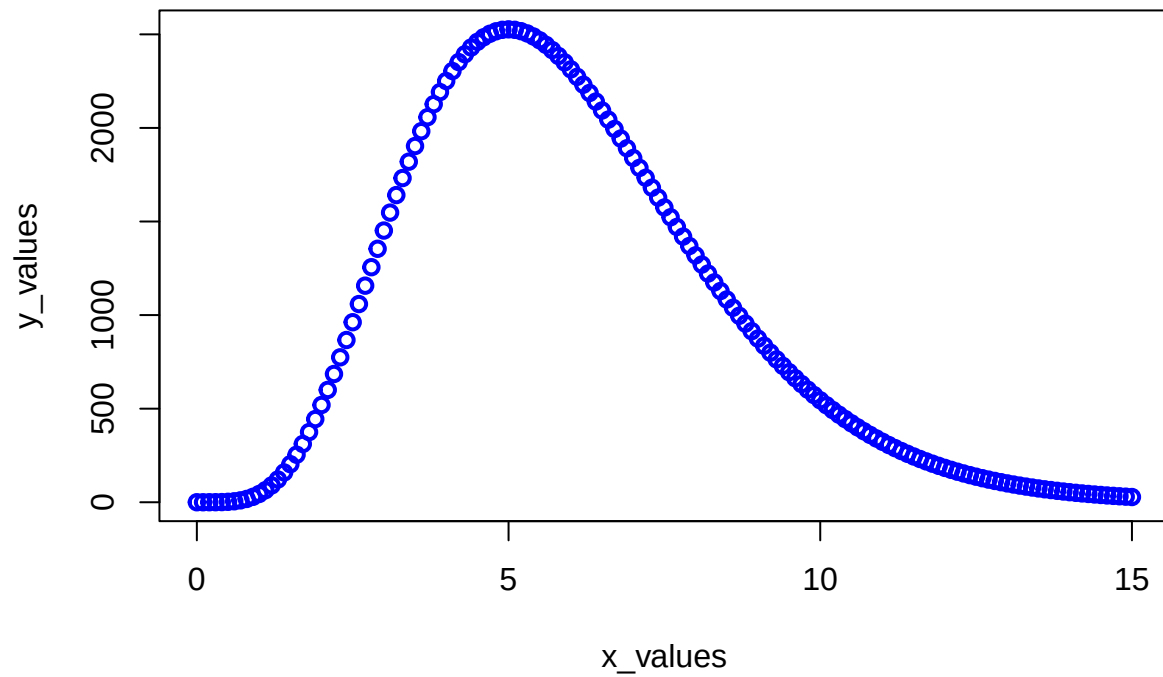
```
##          [,1]
## chain2 1.000000
## x      1.000000
## x      4.419132
## x      8.273999
## x      8.273999
## x      8.273999
```

Metropolis-Hastings Chain



Histogram of Samples with Target Density





Acceptance Rate: 0.6062

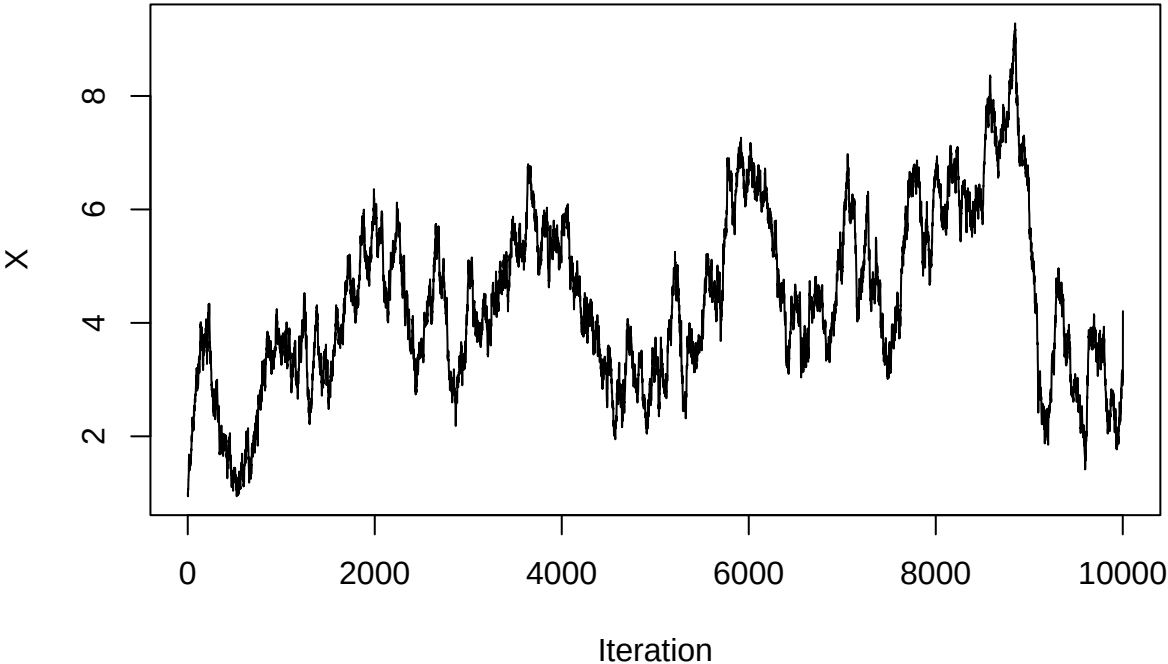
Part c

Suggest another proposal distribution (can be a normal or chi-square distribution with other parameters or another distribution) with the potential to generate a good sample. Perform Part a with this distribution.

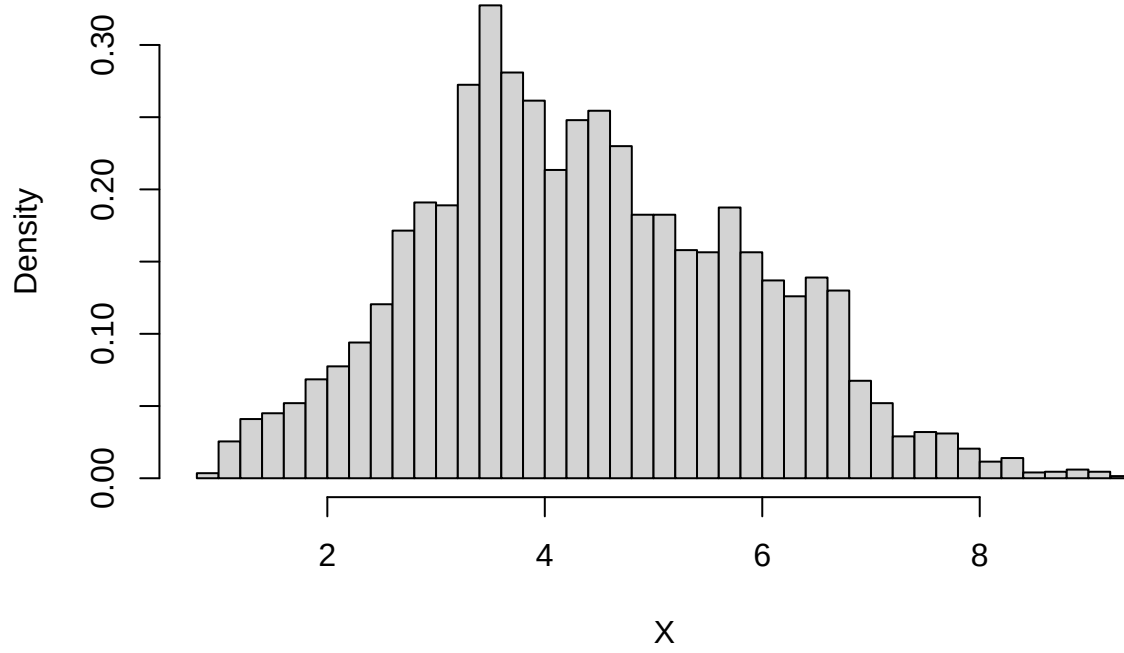
A normal distribution with standard deviation 5 is suggested.

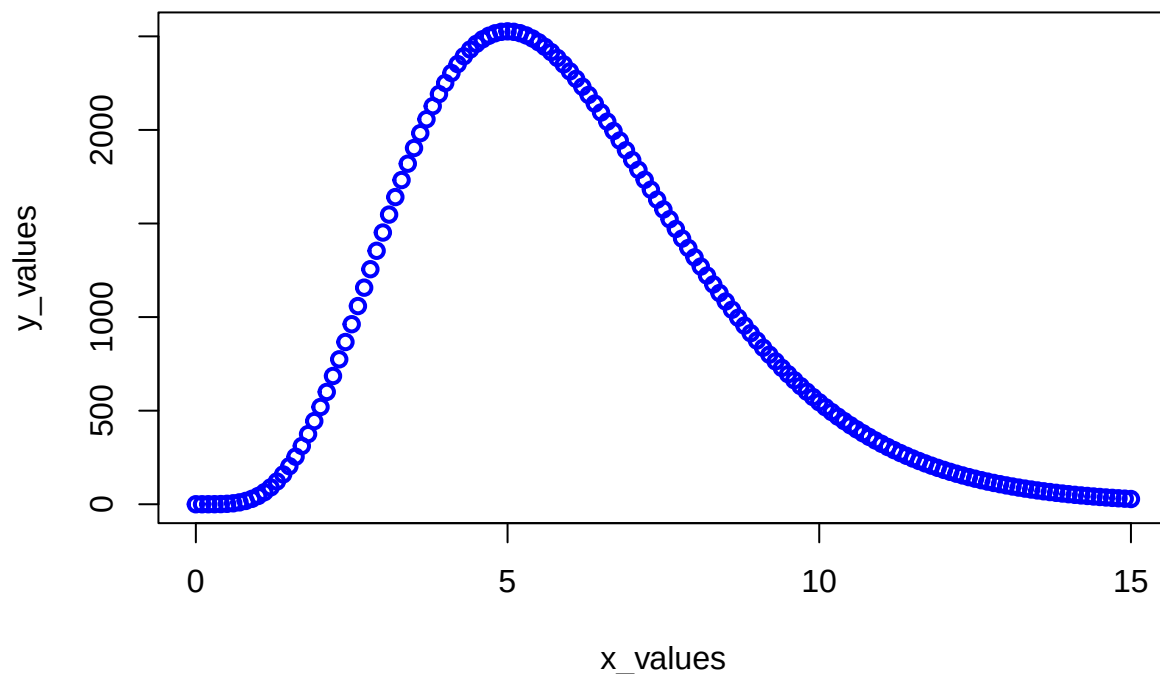
```
##          [,1]
## chain 1.0000000
## x      0.9439524
## x      1.0629731
## x      1.0700239
## x      1.0591273
## x      1.1052190
## x      1.2332744
## x      1.2332744
## x      1.2836557
## x      1.3237328
```

Metropolis-Hastings Chain



Histogram of Samples with Target Density





Acceptance Rate: 0.4737

This chain is generally considered to reach convergence for the stable plot and its good fitting of the target distribution. X-values reaches 2 and begin to stabilize at the 5th iteration, indicating a very short burn-in period. Calculated acceptance rate is 0.4737.

Part d

Compare the results of Parts a, b, and c and make conclusions.

Chain 1 has not yet reached convergence with unstable plot and bad fitting of the true plot. Chain 2 and 3 are considered to reach convergence and they both have good representation of the distribution. While chain 2 has a higher acceptance rate 0.6062 comparing to 0.4737 in chain 3. As is suggest by Hoeting, Chapter 7.3, the preferable acceptance rate is 44%. Therefore, chain 3 is the best out of all 3 chains.

Part e

Estimate

$$E(X) = \int_0^{\infty} xf(x) dx$$

using the samples from Parts a, b, and c.

Since samples follow the distribution $f(x)$, the sample mean is calculated as:

$$\hat{E}(X) = \frac{1}{n} \sum_{i=1}^n X_i$$

where n is the number of samples.

E(X) for Chain 1: 4.383739

E(X) for Chain 2: 5.976023

E(X) for Chain 3: 4.383721

The expectation values are 4.383739 , 5.976023 and 4.383721 for chain 1-3 respectively.

Part f

The distribution generated is in fact a gamma distribution. Search in the literature/internet and define the actual value of the integral. Compare it with the one you obtained.

The given density function is:

$$f(x) = 120x^5e^{-x}, \quad x > 0.$$

A gamma distribution has the probability density function:

$$g(x; \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, \quad x > 0.$$

By comparing with the given function:

- The exponent of x suggests $\alpha = 6$ since $x^5 = x^{6-1}$.
- The exponential term e^{-x} suggests $\beta = 1$.

The normalizing constant is:

$$\frac{1}{\Gamma(6)} = \frac{1}{120}.$$

Thus, the given function is indeed a Gamma distribution with parameters $\alpha = 6$ and $\beta = 1$. Since for Gamma distribution, we have

$$E(X) = \frac{\alpha}{\beta}.$$

$$E(X) = \frac{6}{1} = 6.$$

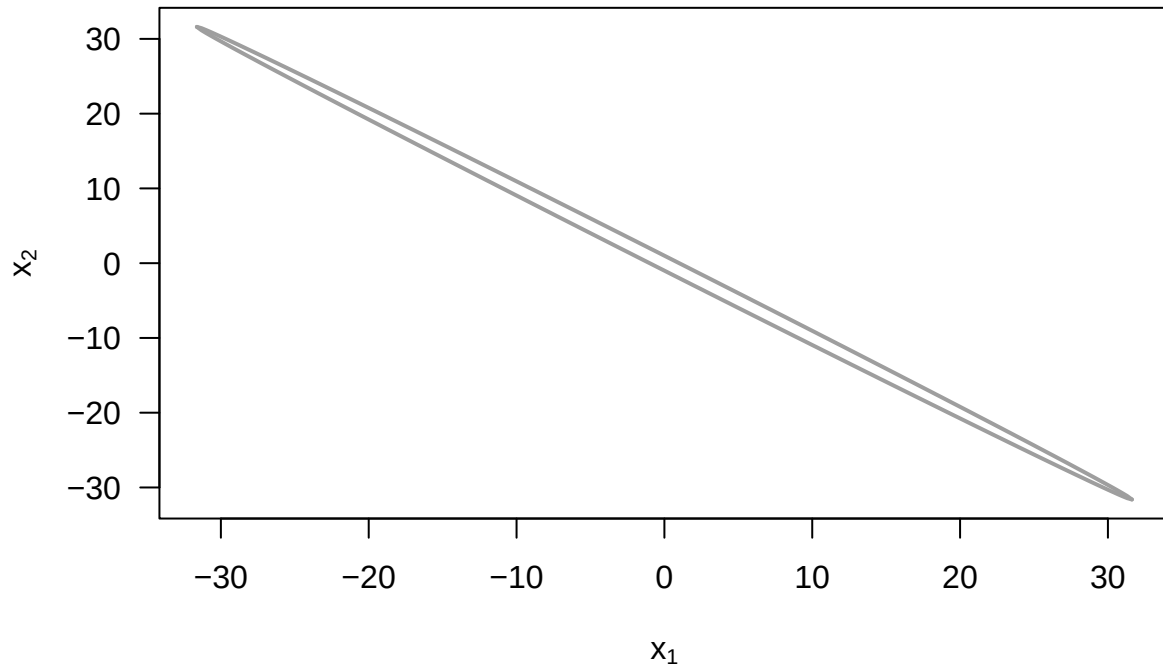
The theoretical expectation of X is 6. Comparing the theoretical expectation with the results from e), chain 2 gives the most close result.

Question 2: Gibbs Sampling

Let $X = (X_1, X_2)$ be a bivariate distribution with density $f(x_1, x_2) \propto 1\{x_1^2 + wx_1x_2 + x_2^2 < 1\}$ for some specific w with $|w| < 2$. X has a uniform distribution on some two-dimensional region. We consider here the case $w = 1.999$ (in Lecture 4, the case $w = 1.8$ was shown).

Part a

Draw the boundaries of the region where X has a uniform distribution. You can use the code provided on the course homepage and adjust it.



Part b

What is the conditional distribution of X_1 given X_2 and that of X_2 given X_1 ? $X_1 \mid X_2$:

$$x_1 \mid x_2 = \frac{-1.999x_2 \pm \sqrt{(1.999x_2)^2 - 4(x_2^2 - 1)}}{2}$$

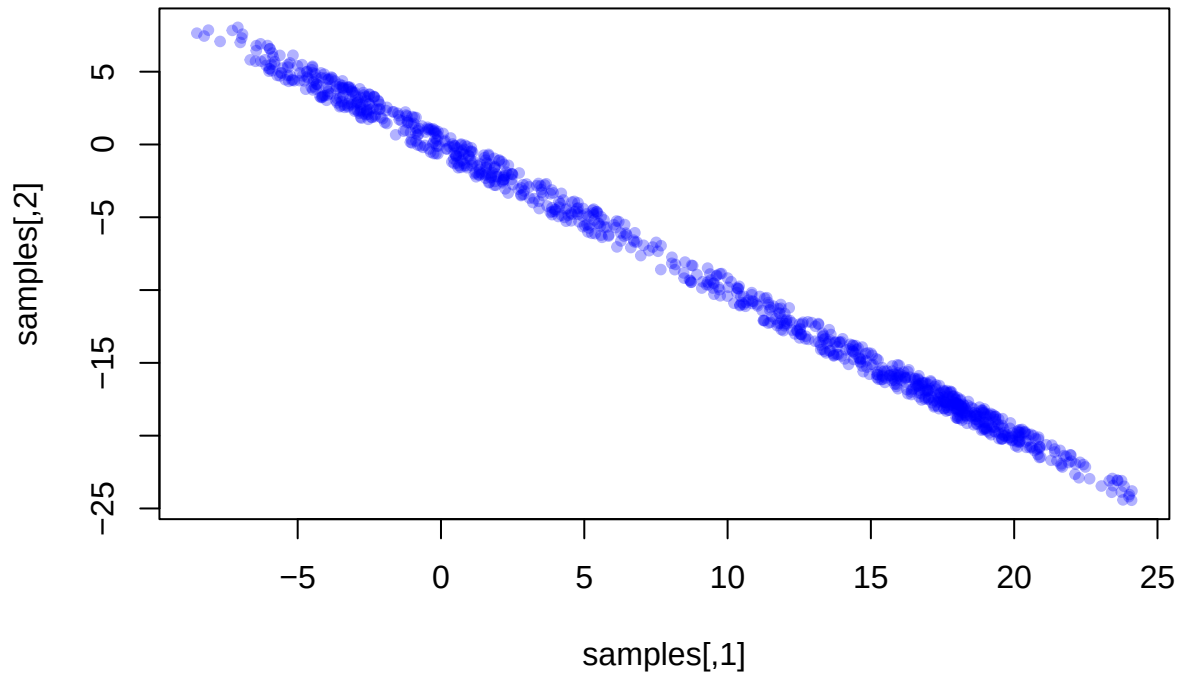
$X_2 \mid X_1$:

$$x_2 \mid x_1 = \frac{-1.999x_1 \pm \sqrt{(1.999x_1)^2 - 4(x_1^2 - 1)}}{2}$$

Part c

Write your own code for Gibbs sampling the distribution. Run it to generate $n = 1000$ random vectors and plot them into the picture from Part a. Determine $P(X_1 > 0)$ based on the sample and repeat this a few times (you need not plot the repetitions). What should be the true result for this probability?

Gibbs Samples



Estimated $P(X_1 > 0)$: 0.755

Part d

Discuss, why the Gibbs sampling for this situation seems to be less successful for $w = 1.999$ compared to the case $w = 1.8$ from the lecture.

From the plot above, we can see that x_1 and x_2 is highly correlated, The ellipse becomes extremely narrow compared to $W = 1.8$ and approaches a line. so the sample value could only in a very narrow range. Discuss why the Gibbs sampling for this situation seems to be less successful for $w = 1.999$ compared to the case $w = 1.8$ from the lecture. We might transform the variable X and generate $U = (U_1, U_2) = (X_1 - X_2, X_1 + X_2)$ instead. In this case, the density of the transformed variable $U = (U_1, U_2)$ is again a uniform distribution on a transformed region (no proof necessary for this claim). Determine the boundaries of the transformed region where U has a uniform distribution on. You can use that the transformation corresponds to $X_1 = \frac{U_2 + U_1}{2}$ and $X_2 = \frac{U_2 - U_1}{2}$ and set this into the boundaries in terms of X_i . Plot the boundaries for (U_1, U_2) . Generate $n = 1000$ random vectors with Gibbs sampling for U and plot them. Determine $P(X_1 > 0) = P\left(\frac{U_2 + U_1}{2} > 0\right)$. Compare the results with Part c.

Part e

We might transform the variable X and generate $U = (U_1, U_2) = (X_1 - X_2, X_1 + X_2)$ instead. In this case, the density of the transformed variable $U = (U_1, U_2)$ is again a uniform distribution on a transformed region (no proof necessary for this claim). Determine the boundaries of the transformed region where U has a uniform distribution on. You can use that the transformation corresponds to:

$$X_1 = \frac{U_2 + U_1}{2}, \quad X_2 = \frac{U_2 - U_1}{2}$$

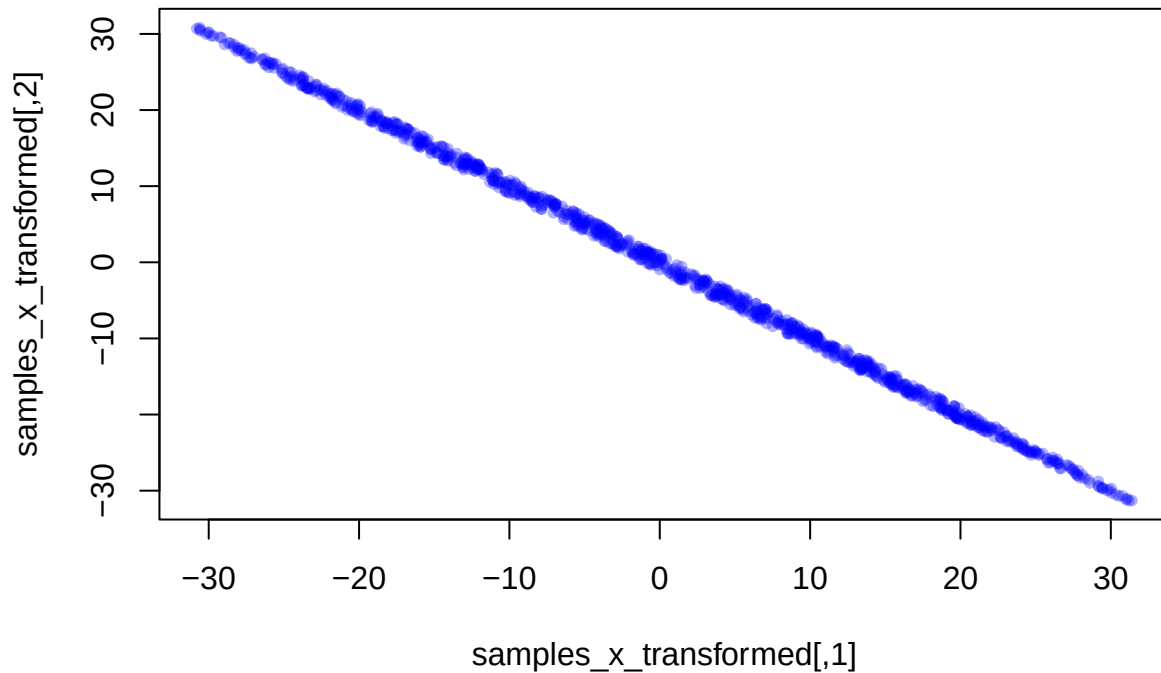
and set this into the boundaries in terms of X_i . Plot the boundaries for (U_1, U_2) . Generate $n = 1000$ random vectors with Gibbs sampling for U and plot them. Determine:

$$P(X_1 > 0) = P\left(\frac{U_2 + U_1}{2} > 0\right).$$

Compare the results with Part c.

Through variable conversion, the value range of X_1 and X_2 is broadened, the sampling efficiency is improved, and the estimation is closer to the true value. But because of w equals 1.999 which results in a slight asymmetry, so the actual value may be closer to 0.5.

Gibbs Samples



```
## Transformed P(X1 > 0): 0.514
```

```
#Assig 1
#a
prop_dis <- function(xt) {
  return(rnorm(1, xt, 0.1))
}

f <- function(x)
{
  return(120 * x ^ 5 * exp(-x))
}
```

```

}

set.seed(123)
x      <- 1
chain  <- c(x)
acceptance_count <- 0

# generate first five values (change 5 to higher number for longer chain)
for (i in 1:10000)
{
  # generate first one sample from uniform distribution on circle with specific
  #radius by rejection sampling:

  xcand <- prop_dis(x)          # candidate point
  # dnorm as the density of it
  R      <- f(xcand) * dnorm(x, mean = xcand, sd = 0.1) /
    (f(x) * dnorm(xcand, mean = x, sd = 0.1)) # MH ratio

  ap      <- runif(1)
  if (ap < R) {
    x <- xcand
    acceptance_count <- acceptance_count + 1
  }

  chain <- rbind(chain, x)
}
head(chain) # each row is one two-dimensional observation

plot(
  chain,
  type = "l",
  main = "Metropolis-Hastings Chain",
  xlab = "Iteration",
  ylab = "X"
)
# Plot histogram of the samples
y_values <- f(x_values)
plot(x_values, y_values, col = "blue", lwd = 2)

# Calculate acceptance rate
acceptance_rate <- acceptance_count / 10000
cat("Acceptance Rate:", acceptance_rate, "\n")

#b

prop_dis2 <- function (x, xt) {
  dchisq(x, df = floor(xt + 1))
}

set.seed(123)

```

```

x      <- 1
chain2 <- c(x)
acceptance_count <- 0
for (i in 1:10000)
{
  # generate first one sample from uniform distribution on circle with specific
#radius by rejection sampling:

  xcand <- rchisq(1, floor(x + 1))      # candidate point
  # dnorm as the density of it
  R      <- f(xcand) * prop_dis2(x, xcand) /
    (f(x) * prop_dis2(xcand, x)) # MH ratio

  ap      <- runif(1)
  if (ap < R) {
    x <- xcand
    acceptance_count <- acceptance_count + 1
  }
  chain2 <- rbind(chain2, x)
}
head(chain2) # each row is one two-dimensional observation

plot(
  chain2,
  type = "l",
  main = "Metropolis-Hastings Chain",
  xlab = "Iteration",
  ylab = "X"
)

# Plot the histogram of the samples
hist(
  chain2,
  breaks = 50,
  main = "Histogram of Samples with Target Density",
  xlab = "X",
  probability = TRUE
)
plot(x_values, y_values, col = "blue", lwd = 2)
# Calculate acceptance rate
acceptance_rate <- acceptance_count / 10000
cat("Acceptance Rate:", acceptance_rate, "\n")

#c
test_deviation = 5
prop_dis3 <- function (xt) {
  return(rnorm(1, xt, test_deviation))
}

f <- function(x)
{

```



```

    return(120 * x ^ 5 * exp(-x))
}

set.seed(123)
x      <- 1
chain3 <- c(x)
acceptance_count <- 0

# generate first five values (change 5 to higher number for longer chain)
for (i in 1:10000)
{
  # generate first one sample from uniform distribution on circle with specific
  # radius by rejection sampling:

  xcand <- prop_dis3(x)      # candidate point
  # dnorm as the density of it
  R      <- f(xcand) * dnorm(x, mean = xcand, sd = test_deviation) /
    (f(x) * dnorm(xcand, mean = x, sd = test_deviation)) # MH ratio

  ap      <- runif(1)
  if (ap < R) {
    x <- xcand
    acceptance_count <- acceptance_count + 1
  }

  chain3 <- rbind(chain, x)
}
head(chain3, 10) # each row is one two-dimensional observation

plot(
  chain3,
  type = "l",
  main = "Metropolis-Hastings Chain",
  xlab = "Iteration",
  ylab = "X"
)
# Plot histogram of the samples

# Plot the histogram of the samples
hist(
  chain3,
  breaks = 50,
  main = "Histogram of Samples with Target Density",
  xlab = "X",
  probability = TRUE
)

# Add the target density function
x_values <- seq(0, 15, by = 0.1)
y_values <- f(x_values)
plot(x_values, y_values, col = "blue", lwd = 2)

```

```

# Calculate acceptance rate
acceptance_rate <- acceptance_count / 10000
cat("Acceptance Rate:", acceptance_rate, "\n")

#e
chian1_exp <- mean(chain)
chian2_exp <- mean(chain2)
chian3_exp <- mean(chain3)

cat("E(X) for Chain 1: ", chian1_exp, "\n")
cat("E(X) for Chain 2: ", chian2_exp, "\n")
cat("E(X) for Chain 3: ", chian3_exp, "\n")

#assig 2
#a
w <- 1.999
xv <- seq(-1, 1, by = 0.01) * 1 / sqrt(1 - w ^ 2 / 4) # a range of x1-values,
#where the term below the root is non-negative (compare Lecture 4)
plot(
  xv,
  xv,
  type = "n",
  xlab = expression(x[1]),
  ylab = expression(x[2]),
  las = 1
)
# ellipse
lines(xv, -(w / 2) * xv - sqrt(1 - (1 - w ^ 2 / 4) * xv ^ 2), lwd = 2, col =
      8)
lines(xv, -(w / 2) * xv + sqrt(1 - (1 - w ^ 2 / 4) * xv ^ 2), lwd = 2, col =
      8)

#c
get_limits <- function(fixed_val, w, var) {
  a <- 1
  b <- w * fixed_val
  c_val <- fixed_val ^ 2 - 1
  discriminant <- b ^ 2 - 4 * a * c_val
  if (discriminant < 0)
    return(c(NA, NA))
  sqrt_d <- sqrt(discriminant)
  lower <- (-b - sqrt_d) / (2 * a)
  upper <- (-b + sqrt_d) / (2 * a)
  return(c(lower, upper))
}

gibbs_sample <- function(n, w) {
  samples <- matrix(0, nrow = n, ncol = 2)
  x1 <- 0
  x2 <- 0
  for (i in 1:n) {

```

```

#browser()
# fix X2 and update X1 / X2
limits_x1 <- get_limits(x2, w, "x1")
if (any(is.na(limits_x1)))
  break
#sample between the lower and upper limits
x1 <- runif(1, limits_x1[1], limits_x1[2])
# fix X2 and update X2 / X1
limits_x2 <- get_limits(x1, w, "x2")
if (any(is.na(limits_x2)))
  break
x2 <- runif(1, limits_x2[1], limits_x2[2])
samples[i, ] <- c(x1, x2)
}
return(samples)
}
set.seed(123)
samples <- gibbs_sample(1000, 1.999)
plot(samples,
      pch = 20,
      col = rgb(0, 0, 1, 0.3),
      main = "Gibbs Samples")

prob <- mean(samples[, 1] > 0)
cat("Estimated P(X1 > 0):", prob)

#e
gibbs_sample_transformed = function(n, w) {
  samples_u = matrix(0, nrow = n, ncol = 2)
  u1 <- 0
  u2 <- 0
  for (i in 1:n) {
    # fix U2 and update U1 / U2
    u2_current <- u2
    u1_min <- -sqrt((4 - 3.999 * u2_current ^ 2) / 0.001)
    u1_max <- sqrt((4 - 3.999 * u2_current ^ 2) / 0.001)
    u1 <- runif(1, u1_min, u1_max)
    # fix U1 and update U2 / U1
    u1_current <- u1
    u2_min <- -sqrt((4 - 0.001 * u1_current ^ 2) / 3.999)
    u2_max <- sqrt((4 - 0.001 * u1_current ^ 2) / 3.999)
    u2 <- runif(1, u2_min, u2_max)
    samples_u[i, ] <- c(u1, u2)
  }
  return(samples_u)
}

samples_u <- gibbs_sample_transformed(1000, 1.999)
#browser()
samples_x_transformed <- cbind((samples_u[, 1] + samples_u[, 2]) / 2,
                              (samples_u[, 2] - samples_u[, 1]) / 2)
plot(
  samples_x_transformed,

```

```
pch = 20,  
col = rgb(0, 0, 1, 0.3),  
main = "Gibbs Samples"  
)  
  
prob_u <- mean(samples_x_transformed[, 1] > 0)  
cat("Transformed P(X1 > 0):", prob_u)
```